

Mục đích:

Kết thúc chương 15 bạn có thể

- Khai báo các bẫy lỗi.
- Liệt kê các lợi ích của bẫy lỗi.
- Hiểu và sử dụng lệnh CREATE TRIGGER T-SQL.
- Mô tả các bảng Inserted và Deleted được sử dụng bởi bẫy lỗi
- Giải thích các kiểu bẫy lỗi: INSERT, UPDATE, và DELETE
- Khai báo bẫy lỗi xếp theo tầng và bẫy lỗi xếp lồng vào nhau.
- Khai báo các bẫy lỗi INSTEAD OF
- Chỉ ra các nhân tố tác động đến việc thực thi hành bẫy lỗi.

Lời giới thiệu

Ở chương trước, chúng ta đã được học về thủ tục lưu và cách sử dụng chúng thành thạo. Chúng ta đã thảo luận về thủ tục lưu hệ thống và các thủ tục lưu do người dùng khai báo. Học cách truyền tham số tới các thủ tục lưu cùng với các tùy chọn biên dịch lại của chúng.

Trong chương này, chúng ta sẽ học về một kiểu thủ tục lưu đặc biệt. Thủ tục này được gọi là Bẫy lỗi. Mỗi một bẫy lỗi được gắn liền với một bảng, và nó được thực thi khi xuất hiện một sự kiện đặc biệt trong bảng. Chúng ta sẽ thảo luận về một vài kiểu bẫy lỗi, cách thực thi, và các giới hạn của chúng.

15.1 Các bẫy lỗi

Các bẫy lỗi là các thủ tục lưu, chúng được thực thi để phản ứng lại các thao tác chèn, cập nhật, hoặc xóa trong một bảng. Các bẫy lỗi được kích hoạt một cách tự động khi có một trong các thao tác trên xuất hiện. Các bẫy lỗi được sử dụng một cách phổ biến để ép các thao tác tuân theo các quy tắc nhất định. Chúng giám sát sự thay đổi và để chắc chắn rằng những sự thay đổi đó phù hợp với các quy tắc công việc được thiết lập. Vì vậy, các bẫy lỗi đảm bảo tính toàn vẹn và tính nhất quán của dữ liệu.

Các bẫy lỗi là các đối tượng đặc biệt được tạo ra trong một bảng, và là một phần của cơ sở dữ liệu. Một bẫy lỗi có thể truy vấn đến nhiều bảng khác nhau, và có thể bao gồm các câu lệnh T-SQL phức tạp. Chúng ta có thể tạo các bẫy lỗi riêng biệt cho mỗi thao tác đối với một bảng, hoặc kết hợp 2 hoặc 3 thao tác cho một bẫy lỗi. Trong trường hợp của nhiều bẫy lỗi, chúng ta bắt buộc phải sắp xếp thứ tự mà chúng được thực thi.

15.1.1 Mục đích của các bẫy lỗi

Một số mục đích thông thường của bẫy lỗi như sau:

- So sánh kiểu dữ liệu.
- Đọc dữ liệu từ các bảng nằm trong cơ sở dữ liệu khác.
- Thay đổi theo tầng hoặc xóa liên tục các bảng liên quan trong một cơ sở dữ liệu
- Hủy bỏ các thay đổi không đúng
- Tuân theo các giới hạn, các giới hạn đó phức tạp hơn việc bắt lỗi bằng ràng buộc CHECK
- Thực thi các thủ tục lưu cục bộ và các thủ tục lưu từ xa.

15.2 Tạo các bẫy lỗi

Các bẫy lỗi có thể được tạo bằng công cụ Enterprise Manager, hoặc Query Analyzer. Trong cả hai trường hợp, câu lệnh CREATE TRIGGER được sử dụng để tạo ra bẫy lỗi. Câu lệnh này nhận biết bảng được sử dụng để khai báo bẫy lỗi, và các sự kiện mà bẫy lỗi sẽ được gọi đến. Cú pháp của câu lệnh :

```
CREATE TRIGGER Trigger_name
ON table
FOR [DELETE, INSERT, UPDATE]
[WITH ENCRYPTION]
AS Sql_statements
```

Trong đó:

Trigger_name: Tên của bẫy lỗi.

table: Tên của bảng mà bẫy lỗi sẽ được thực thi.

WITH ENCRYPTION: Tùy chọn này được sử dụng để ngăn cản người sử dụng có thể xem được mã của bẫy lỗi.

[DELETE, INSERT, UPDATE]: Các câu lệnh đặc biệt, các câu lệnh này sẽ gọi đến bẫy lỗi .

Sql_statements: câu lệnh T-SQL đặc biệt được thực thi khi bẫy lỗi được gọi đến.

15.2.1 Hướng dẫn tạo các bẫy lỗi

Hãy xem xét một số điểm trước khi tạo các bẫy lỗi:

- Một bẫy lỗi có thể kết hợp ba thao tác thực hiện trên một bảng như INSERT, UPDATE, và DELETE.
- Một bẫy lỗi thích ứng với một bảng đơn.
- Tùy chọn WITH ENCRYPTION có thể được sử dụng để làm ẩn các khai báo của một bẫy lỗi đối với người sử dụng. Tuy nhiên, một bẫy lỗi đã được mã hoá sẽ không thể bị giải mã
- Một bẫy lỗi có thể tham chiếu đến một view hoặc một bảng tạm thời, nhưng không thể kết hợp với chúng.
- Một bẫy lỗi có thể bao gồm các câu lệnh SQL với số lượng bất kì.
- Một số các câu lệnh SQL không thể có trong bẫy lỗi như: CREATE DATABASE, ALTER DATABASE, DROP DATABASE, LOAD DATABASE, RECONFIGURE, RESTORE LOG, DISK INIT, DISK RESIZE, LOAD LOG, RESTORE DATABASE
- Mặc định, chỉ có người tạo ra cơ sở dữ liệu đó mới có quyền tạo một bẫy lỗi trong nó. Quyền này không thể chuyển giao.
- Một bẫy lỗi chỉ có thể được tạo trong cơ sở dữ liệu hiện hành. Tuy nhiên, bẫy lỗi có thể tham chiếu đến các đối tượng nằm trong cơ sở dữ liệu khác.

15.3 Sử dụng bảng Inserted và bảng Deleted

Đoạn mã chứa trong một bẫy lỗi có thể truy cập đến tới hai bảng logic. Các bảng này là bộ phận của bẫy lỗi, và chúng được gọi là **Inserted** và **Deleted**. Bảng **Inserted** và **Deleted** chứa ảnh của dữ liệu trước và sau quá trình cập nhật. Giả sử đồ tương đồng với bảng đã được cập nhật. Dữ liệu trong bảng không bị tác động bởi thao tác cập nhật sẽ không nằm trong bảng **Inserted** và **Deleted**.

Bảng 15.1 chỉ ra vai trò của các bảng **Inserted** và **Deleted** trong quá trình thực thi các bẫy lỗi INSERT, UPDATE and DELETE .

Kiểu bẫy lỗi	Bảng <i>Inserted</i>	Bảng <i>Deleted</i>
UPDATE	Chứa một bản sao của các bản ghi đã được cập nhật sau khi câu lệnh được thực thi xong.	Chứa một bản sao nguyên bản của các bản ghi đã được cập nhật sau khi câu lệnh được thực thi xong..
DELETE	Không sử dụng.	Chứa các bản ghi đã được xoá sau khi câu lệnh thực thi xong.
INSERT	Chứa một bản sao của các bản ghi đã được chèn thêm sau khi câu lệnh được thực thi xong	Không sử dụng.

Bảng 15.1: Vai trò của bảng **Inserted** và bảng **Deleted** trong bẫy lỗi

15.4 Các kiểu bẫy lỗi

en

Một vài kiểu bẫy lỗi sẽ được mô tả chi tiết dưới đây .

15.4.1 Bẫy lỗi INSERT

Một bẫy lỗi INSERT được thực thi bất cứ khi nào có thao tác chèn thêm dữ liệu vào một bảng. Bẫy lỗi INSERT để chắc chắn rằng dữ liệu được thêm vào bảng là hợp lệ. Dữ liệu chèn thêm vào bảng được lưu giữ ở trong bảng logic **Inserted**, và được thêm vào bảng có bẫy lỗi. Bảng logic **Deleted** không bị sử dụng trong thao tác chèn thêm dữ liệu, bởi vì không có dữ liệu nào bị xóa.

Bẫy lỗi INSERT thực thi các thao tác dưới đây khi mà có một đối tượng được tạo ra để chèn thêm dữ liệu vào bảng:

1. Chèn một bản sao của hàng mới vào bảng **Inserted**.
2. Kiểm tra hàng mới trong bảng **Inserted**, để xác định xem dữ liệu được chèn vào có hợp lệ hay không.
3. Nếu tìm thấy các giá trị trong hàng chèn vào phù hợp điều kiện chỉ ra, chèn chúng vào trong bảng bẫy lỗi.

Ví dụ, một bẫy lỗi INSERT có tên là **CheckRate** được tạo trên bảng **Item** của cơ sở dữ liệu **StrongHold**. Bẫy lỗi đảm bảo rằng đơn giá cho bất kỳ sản phẩm nào thêm vào đều không vượt qua 1000. Mã cho bẫy lỗi như sau:

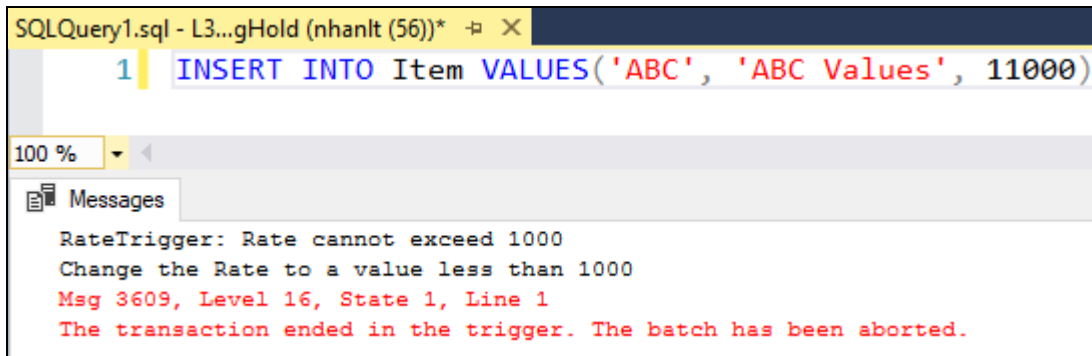
```
CREATE TRIGGER CheckRate
ON Item
FOR INSERT AS
IF (SELECT Rate FROM inserted) > 1000
```

```

BEGIN
    PRINT 'RateTrigger: Rate cannot exceed 1000'
    PRINT 'Change the Rate to a value less than 1000'
    ROLLBACK TRANSACTION
END

```

Để kiểm tra công việc của bẫy lỗi INSERT, chúng ta cùng thêm mới một hàng, như hình 15.1. Hình này cũng hiển thị kết quả của tác vụ. Trong trường hợp này, hàng không được chèn vào trong bảng bởi vì điều kiện đưa ra trong trigger (đơn giá không quá 1000) là không phù hợp. Giao dịch được trả lại và hiển thị thông báo lỗi tới người sử dụng.



Hình 15.1: Bẫy lỗi INSERT

15.4.2 Bẫy lỗi UPDATE

Một bẫy lỗi UPDATE được thực hiện khi một tác vụ cập nhật thực thi trên bảng. Nó thực thi các tác vụ dưới đây:

1. Di chuyển hàng dữ liệu gốc vào bảng logic **Deleted**.
2. Chèn một hàng mới vào bảng **Inserted**, và bảng bẫy lỗi.
3. Tính toán các giá trị trong các bảng **Deleted** và **Inserted** để xác định can thiệp cần thiết.

Các bẫy UPDATE có thể được tạo ra để xác nhận việc cập nhật trên một cột đơn, hoặc trên toàn bộ bảng.

➤ **Mức cột** – Bẫy lỗi xuất hiện khi dữ liệu trên cột chỉ định được cập nhật.

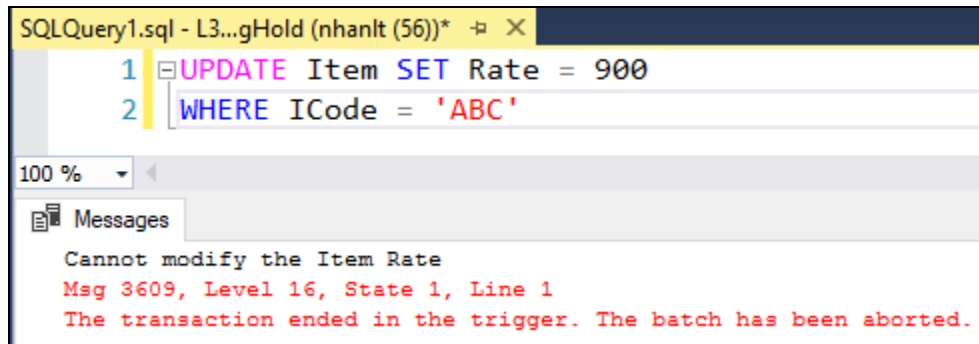
Ví dụ, một bẫy UPDATE được tạo để đảm bảo rằng cột **Rate** trong bảng **Item** không sửa đổi được. Bẫy này sẽ được thực thi mỗi khi tác vụ cập nhật xảy ra trên cột **Item**. Mã của của bẫy là:

```

CREATE TRIGGER NoUpdateRate
ON Item
FOR UPDATE AS
IF UPDATE (Rate)
BEGIN
    PRINT 'Cannot modify the Item Rate'
    ROLLBACK TRANSACTION
END

```

Sau khi bẫy lỗi được tạo ra, không một tác vụ cập nhật nào được phép trên cột **Rate**. Hình 15.2 mô tả tác động của việc cập nhật trên cột **Rate** trong bảng **Item**.



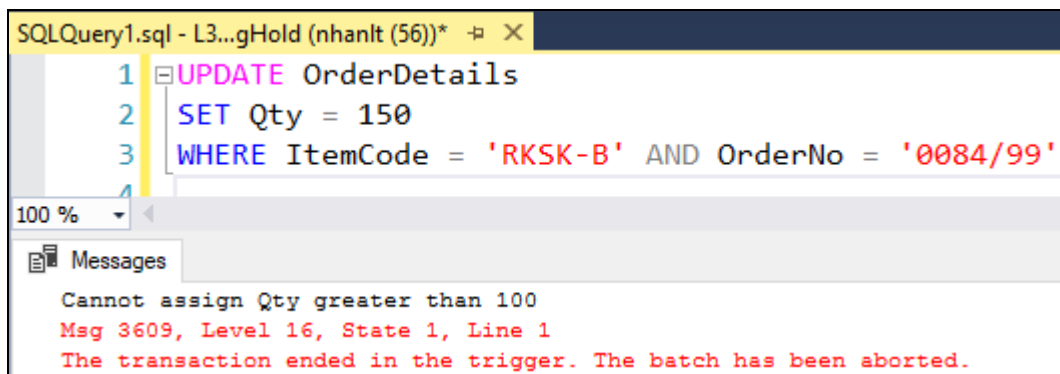
Hình 15.2: Bẫy UPDATE ở mức cột

- **Mức bảng** – Bẫy này được gọi khi bất kỳ một trường nào trong hàng của bảng được cập nhật.

Ví dụ, một bẫy UPDATE được tạo để đảm bảo rằng cột **Qty** không được sửa đổi nếu như giá trị của nó lớn hơn 100. Bẫy lỗi được thực thi mỗi khi cập nhật cột trong bảng **OrderDetails**. Mã của bẫy là:

```
CREATE TRIGGER NoUpdateOrderDetails
ON OrderDetails
FOR UPDATE AS
IF (SELECT Qty FROM inserted) > 100
BEGIN
    PRINT 'Cannot assign Qty greater than 100'
    ROLLBACK TRANSACTION
END
```

Sau khi bẫy **NoUpdateOrderDetails** được tạo, nó sẽ được gọi khi bất kỳ một tác vụ cập nhật nào xảy ra trên bảng **OrderDetails**. Nếu tác vụ đòi hỏi việc cập nhật cột **Qty**, các giá trị sẽ được xác nhận trước khi chèn vào. Hình 15.3 mô tả tác động của việc cập nhật trên bảng **OrderDetails**. Nếu giá trị Qty là 150, lớn hơn 100, việc cập nhật sẽ không được thực hiện.



Hình 15.3: Bẫy lỗi UPDATE mức bảng

15.4.3 Bẫy lỗi DELETE

Một bẫy DELETE được thực hiện mỗi khi xoá các hàng từ một bảng. Vì các hàng không được chèn vào bảng đích, bảng logic **Inserted** không được sử dụng với bẫy DELETE.

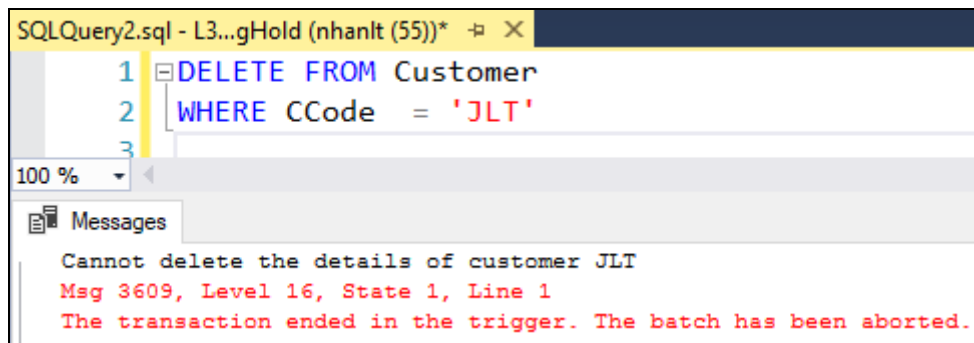
Bẫy DELETE thực hiện các công việc sau:

1. Xoá các hàng từ bảng bẫy lỗi.
2. Chèn các hàng được xoá vào bảng **Deleted**.
3. Kiểm tra các hàng trong bảng **Deleted** để xác định các tác vụ bẫy sẽ được thực hiện như thế nào.

Ví dụ, một bẫy lỗi DELETE được tạo để đảm bảo rằng thông tin chi tiết của nhà xuất bản TLT không được xoá khỏi bảng Customer. Mã bẫy là:

```
CREATE TRIGGER NoDeleteJLT
ON Customer
FOR DELETE AS
IF (SELECT CCode FROM deleted) = 'JLT'
BEGIN
    PRINT 'Cannot delete the details of customer JLT'
    ROLLBACK TRANSACTION
END
```

Hình 15.4 mô tả tác động của việc xoá trên bảng **Customer**. Hàng có Ccode = 'JLT' không được xoá như đã chỉ ra trong bẫy lỗi và thông báo lỗi tương ứng được hiển thị với người sử dụng.



Hình 15.4: Bẫy DELETE

15.5 Bẫy lỗi dây chuyền (Cascading Triggers)

Các bẫy lỗi dây chuyền được sử dụng để bắt buộc sự toàn vẹn trong tham chiếu. Khi một thay đổi xuất hiện trong một bảng, các bẫy lỗi dây chuyền sửa đổi dữ liệu trong các bảng liên quan. Tuy nhiên, các bẫy lỗi không thể được sử dụng để thực hiện việc cập nhật dây chuyền hoặc xoá mà ảnh hưởng đến các ràng buộc của các khoá chính và khoá ngoại. Các bẫy lỗi được thực hiện sau khi các ràng buộc được kiểm tra; nếu một tác vụ bỏ qua một ràng buộc, bẫy không được thực thi.

Chúng ta cùng xét một ví dụ để hiểu về bẫy dây chuyền.

Giả sử chúng ta phải xoá một hàng lưu trữ thông tin về các sản phẩm từ bảng *Item* của CSDL *StrongHold*. Bảng *OrderDetails* chứa các thông tin về đặt hàng của các sản phẩm trên. Bây giờ, khi xoá sản phẩm *Item*, chúng ta cũng muốn xoá tất cả các thông tin về đặt hàng liên quan đến các

sản phẩm đó từ bảng *OrderDetails*. Lưu ý bảng *OrderDetails* có một khoá ngoại ràng buộc trên bảng *Item*. Bấy dưới đây sẽ giúp chúng ta làm điều đó:

```
CREATE TRIGGER DeleteOnItem
ON Item
FOR DELETE AS
BEGIN
    DELETE FROM OrderDetails
    FROM deleted INNER JOIN OrderDetails
    ON OrderDetails.ItemCode = deleted.ICode
END

GO

DELETE FROM Item WHERE ICode = 'RKSK-B'

GO
```

Bẫy lỗi trên sẽ không thành công vì khoá ngoại ràng buộc giữa bảng *OrderDetails* và *Item*. Vì đã có dữ liệu đặt hàng cho sản phẩm đó nên một thông báo lỗi được sinh ra để báo rằng bạn không thể xoá. Khoá ngoại đảm bảo rằng một sản phẩm không tồn tại sẽ không có dữ liệu trong *OrderDetails* được.

Khi bạn cố gắng xoá dữ liệu, bẫy lỗi được gọi sau khi tác vụ thực thi. Nhưng vì khoá ngoại được kiểm tra trước khi bẫy được gọi, bẫy lỗi sẽ không được cho phép thực thi. Tuy nhiên, nó có thể được thực hiện khi khoá ngoại được loại bỏ.

Lỗi sẽ phát sinh khi có khoá ngoại:

```
Msg 547, Level 16, State 0, Line 13
The DELETE statement conflicted with the REFERENCE constraint
"FK_ICode". The conflict occurred in database "StrongHold", table
"dbo.OrderDetails", column 'ItemCode'.
The statement has been terminated.
```

Sẽ thực hiện được nếu muốn cố gắng kiểm tra thực thi khi viết:

```
ALTER Table OrderDetails
DROP CONSTRAINT FK_ICode
GO
DELETE FROM Item WHERE ICode = 'RKSK-B'
```

15.6 Bẫy lỗi lồng nhau

Các bẫy lỗi lồng nhau khi một tác vụ trong một bẫy này khởi tạo một bẫy khác, và tới lượt nó có thể lại khởi tạo một bẫy khác nữa và cứ như vậy. Các bẫy lỗi có thể lồng nhau tới 32 mức.

Để cho phép lồng các bẫy lỗi, chúng ta sử dụng thủ tục lưu hệ thống **sp_configure** như sau:

```
sp_configure 'nested trigger', 1
```

Khi tùy chọn được thiết lập, nó có thể áp dụng cho không chỉ cơ sở dữ liệu hiện tại, mà còn cho tất cả các cơ sở dữ liệu trên server bởi vì nó là tùy chọn mức server, không phải là tùy chọn ở mức cơ sở dữ liệu. Để không cho phép các bẫy lồng nhau, chúng ta thực hiện lệnh dưới đây:

```
sp_configure 'nested trigger', 0
```

Các bẫy lồng nhau thực hiện các chức năng “giữ nhà”, chẳng hạn như, lưu trữ các bản sao lưu của dữ liệu mà một bẫy lỗi trước đó tác động lên.

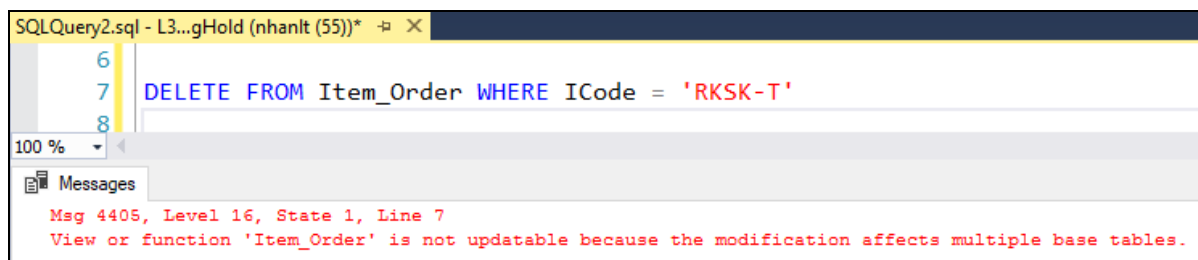
15.7 Bẫy lỗi thay thế (INSTEAD OF)

Các bẫy mà chúng ta vừa bàn đến được thực thi tự động sau khi các tác vụ tác động lên cơ sở dữ liệu. Một bẫy INSTEAD OF chứa đoạn mã thay thế các lệnh thao tác với cơ sở dữ liệu gốc. Các bẫy INSTEAD OF hữu dụng nhất khi việc sửa đổi dữ liệu được thực thi trên view mà chúng không thể cập nhật theo cách thông thường. Các bẫy INSTEAD OF có thể dựa trên chỉ một tác vụ sửa đổi dữ liệu.

Ví dụ, chúng ta đã biết rằng một view dựa trên điều kiện Join không hỗ trợ việc xóa dữ liệu. Bẫy INSTEAD OF cung cấp một giải pháp cho vấn đề này. Đầu tiên chúng ta cùng tạo ra một view dựa vào các bảng **Item** và **OrderDetails** như dưới đây:

```
CREATE VIEW Item_Order
AS
    SELECT B.Icode, B.ItemName, A.Qty
    FROM OrderDetails A, Item B
    WHERE A.ItemCode = B.ICode
```

Lệnh dưới đây thực hiện việc xóa dữ liệu trên view **Item_Order**:



Hình 15.4: Bẫy Item_Order

Để cho phép cập nhật view **Item_Order**, chúng ta phải sử dụng bẫy INSTEAD OF để xoá dữ liệu. Bẫy INSTEAD OF có thể được định nghĩa như sau:

```
CREATE TRIGGER DelOnView
ON Item_Order
INSTEAD OF DELETE
AS
    DELETE FROM Item WHERE ICode IN
        (SELECT ICode FROM DELETED)
```

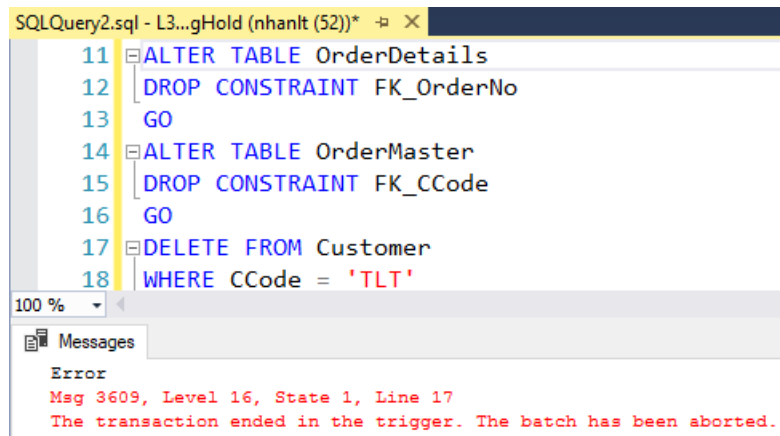
Chạy lại lệnh xoá trước đó: `DELETE FROM Item_Order WHERE ICode = 'RKSK-T'`
Lúc này, dữ liệu được xoá khỏi bảng cơ sở **Item**.

15.8 Bẫy lỗi Master - Details

Ví dụ, bẫy lỗi ngăn việc xoá dữ liệu khách hàng khai khách hàng này vẫn còn đơn đặt hàng:

```
CREATE VIEW testMasterDetails
ON Customer
FOR DELETE AS
    IF EXISTS (SELECT 'true' FROM deleted
                WHERE CCode IN (SELECT A.CCode FROM deleted A
                                INNER JOIN OrderMaster B ON A.CCode = B.CCode))
    BEGIN
        PRINT 'Cannot delete this customer'
        ROLLBACK TRANSACTION
    END
```

Phải bỏ khoá ngoại trước khi test:



Hình 15.4: Bẫy Item_Order

15.9 Bẫy lỗi và hiệu suất

Về mặt hiệu suất thực thi, chi phí dành cho các bẫy lỗi là nhỏ. Hầu hết thời gian đòi hỏi để chạy một bẫy lỗi là dành cho việc tham chiếu đến các bảng khác, có thể nằm trong các thiết bị cơ sở dữ liệu. Các bảng **Deleted** và **Inserted** luôn có mặt trong bộ nhớ. Thời gian yêu cầu cho các tác vụ được xác định bởi vị trí của các bảng mà bẫy tham chiếu đến.

Tổng kết

- Các bẫy lỗi là các thủ tục lưu mà được thực hiện tự động để tương tác với các tác vụ thêm, cập nhật và xóa trên một bảng.
- Các bẫy thường được sử dụng để thực hiện các quy tắc nghiệp vụ đòi hỏi.
- Lệnh CREATE TRIGGER được sử dụng để tạo một bẫy.
- Các bẫy truy nhập tới các bảng logic **Inserted** và **Deleted**. Các bảng này chứa các hình ảnh của dữ liệu trước đó, và sau quá trình cập nhật.
- Các kiểu bẫy:
 - INSERT: Thực hiện mỗi khi xuất hiện việc thêm mới dữ liệu vào bảng. Các bẫy này đảm bảo rằng dữ liệu được chèn vào bảng là hợp lệ.
 - UPDATE: Thực hiện khi một tác vụ cập nhật xảy ra trên một bảng. Các bẫy này có thể được thi hành ở mức bảng hoặc mức cột.
 - DELETE: Thực hiện khi dữ liệu được xóa khỏi một bảng.
- Các bẫy dây chuyền sửa đổi dữ liệu trong các bảng liên quan, nếu một thay đổi xuất hiện trong một bảng.
- Các bẫy lỗi lồng nhau khi một tác vụ trong một bẫy này khởi tạo một bẫy khác, và tới lượt nó có thể lại khởi tạo một bẫy khác nữa và cứ như vậy.
- Thủ tục lưu hệ thống sp_configure được sử dụng để cho phép hoặc không cho phép lồng các bẫy lỗi.
- Bẫy lỗi INSTEAD OF chứa đoạn mã thay thế các lệnh thao tác với dữ liệu gốc.
- Các bẫy INSTEAD OF hữu dụng khi việc sửa đổi dữ liệu được thực hiện trên view mà chúng không thể cập nhật theo cách thông thường.
- Thời gian đòi hỏi để thực thi một bẫy chủ yếu dành nhiều cho việc tham chiếu đến các bảng khác hơn là dành cho các bảng logic.

Kiểm tra kiến thức

1. Kiểu bất lỗi nào dưới đây được thực hiện thay cho lệnh cập nhật gốc?
 - a. Cascade
 - b. INSTEAD OF
 - c. Table level
 - d. Column level
2. Các bất có thể lồng nhau tới bao nhiêu mức?
 - a. 10
 - b. 16
 - c. 32
 - d. 256
3. Tham số WITH ENCRYPTION trong lệnh CREATE TRIGGER dùng để làm gì?
 - a. Mã hoá các đoạn mã của bất lỗi
 - b. Mã hoá cơ sở dữ liệu khi một bất được định nghĩa
 - c. Mã hoá dữ liệu trong cơ sở dữ liệu khi bất được định nghĩa
4. Các bất dây chuyền DELETE hoặc UPDATE mà tác động lên các khoá ngoài được thực hiện trước khi các ràng buộc tương tự và có sẵn trong hệ thống được kiểm tra.
 - a. Đúng
 - b. Sai
5. Các bất lỗi sử dụng các bảng logic nào dưới đây?
 - a. temp
 - b. deleted
 - c. hold
 - d. inserted
6. Bất UPDATE kiểm tra thay đổi trên các đối tượng nào dưới đây?
 - a. A database
 - b. A table
 - c. A column
 - d. A row
7. Bạn tạo một bất **NoUpdate** trên bảng **city** với đoạn mã sau:

```
CREATE TRIGGER NoUpdate
ON city
FOR UPDATE AS
IF (SELECT population FROM inserted) < 5
BEGIN
.....
END
```

Trong trường hợp nào dưới đây, bất sẽ được thực hiện?

- a. Khi cột **population** được cập nhật
- b. Khi cột **city** được cập nhật
- c. Khi bất cứ cột nào trong bảng **population** được cập nhật
- d. Khi bất cứ cột nào trong bảng **city** được cập nhật

8. Lệnh nào dưới đây có thể được bao gồm trong các bẫy lỗi?
- a. ROLLBACK TRANSACTION
 - b. DISK RESIZE
 - c. LOAD DATABASE
 - d. RESTORE LOG

Luyện tập

Viết lệnh Transact-SQL để làm các công việc sau sử dụng cơ sở dữ liệu pubs:

1. Tạo một bẫy lỗi có tên **Royalty_Per_Update** mà không cho phép cập nhật cột **royaltyper** của bảng **titleauthor**.
2. Xem các bẫy lỗi định nghĩa trong bảng **titleauthor**.

This page has been intentionally left blank.